DATA VERSION CONTROL (DVC)

WHY?

✅ Artifacts in ML Pipeline

Artifact = Output of a single block in the ML pipeline.

Examples:

✅ The issue: We treat the ML pipeline as STATIC, but in real life it is NOT.

In practice, **every stage keeps changing**.

📌 Let's continue your example

🎯 Example Situation:

You run a pipeline today:

- Data collected today
- Model trained on today's data

- You get **92% accuracy**
- You save the artifact: *best accuracy model*

Tomorrow:

- New data arrives
- Cleaned data changes
- Features change
- Trained model changes
- New accuracy might be **88% or 95%**

Now your original artifact (best model) is **no longer valid**.

🔥 The Problem

If the pipeline is treated as static:

- You overwrite old artifacts
- You lose track of "which data created this model?"
- Reproduction becomes impossible
- Debugging becomes very hard
- You cannot compare experiments
- You cannot rollback to a previous working version

🧠 Why this happens?

Because in real life:

- Data is dynamic
- Hyperparameters change
- Code changes
- Model architecture changes
- Real-world input keeps updating

So the **pipeline output will always change**.

🚀 Conclusion

> **ML pipelines must be dynamic, versioned, and reproducible.**
> **Because as data, code, or parameters change → every artifact changes.**

By treating the pipeline we means if any thing changed the the pipline the current flow does not change means if we data by collection is updated we dont have to chnge the Feature eng processs, but actually we hav to re-do it so we have correct artifact . In simple if any step is updated the pipline has to be updated

Example 2: Now we have done some chnages in Feature eng , lets say we have initlly did feature eng on bases of variacne threshold , but now we have switch to backward chaning ,it means the artifact of feature eng block changes too that should be consider too so the input to Model training goes correctly

Data versioning is needed because data keeps changing, and you must know which data was used to train which model.

That's it.

🔍 Why data changes?

- New rows get added
- Mistakes are corrected
- Dataset is cleaned differently
- New features are added

Every time data changes, the **model output also changes**.

🧠 Without data versioning:

- You **cannot reproduce** past results.
- You **don't know which dataset** created a good model.
- You **can't compare** old vs new experiments.
- Team members have **different versions** of the data → confusion.

👍 With data versioning:

- You always know **what changed** and **when**.
- You can rebuild a model exactly as before.
- You can compare experiments fairly.
- No data confusion across team members.

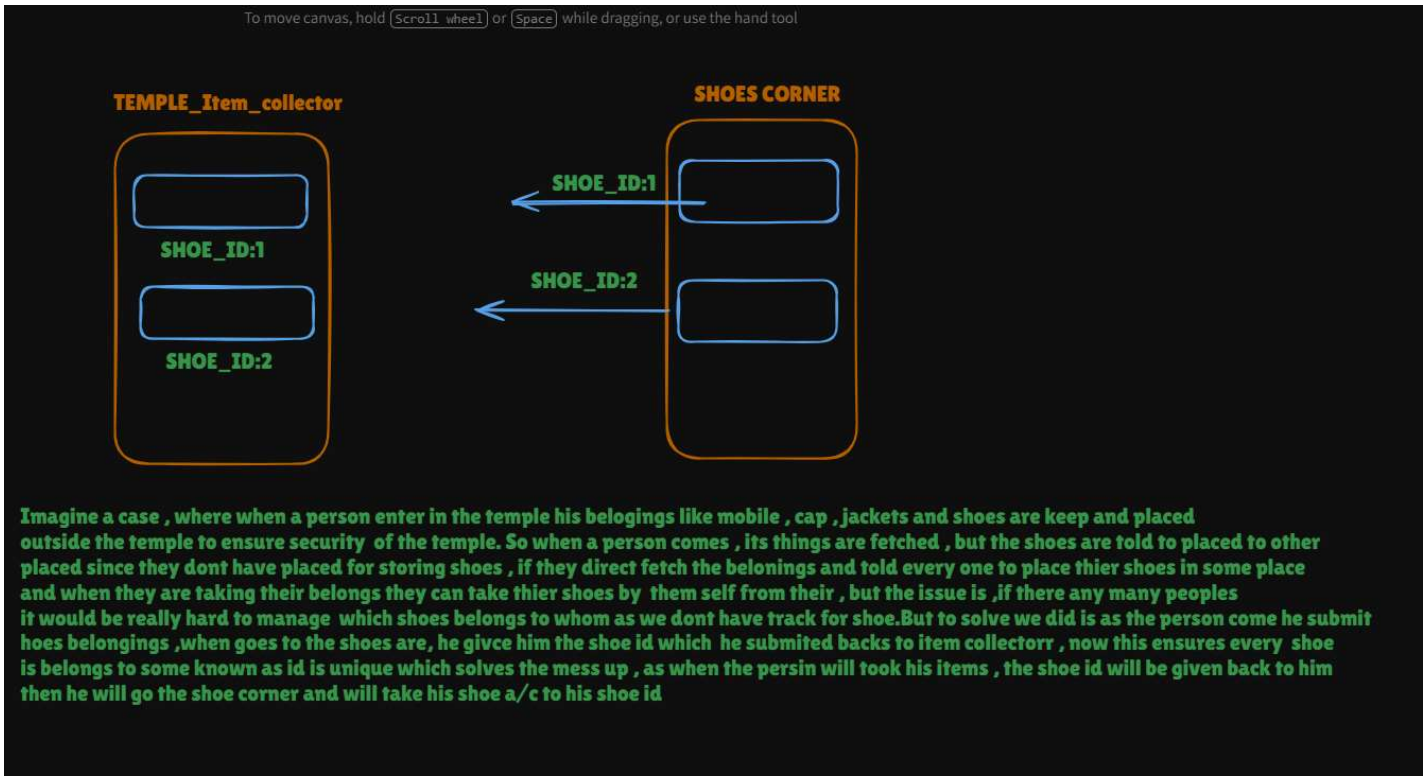
🔑 Simple explanation:

> **Just like code has versions, data must also have versions—because ML models depend on data.**

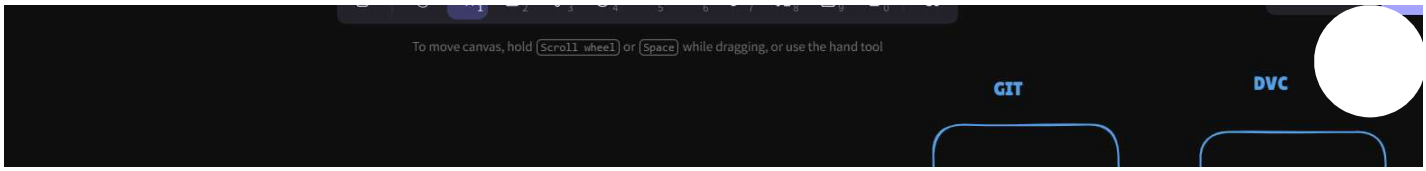
Here the data does not only means the tabular data coming from some source but also the data from artifact for example in trial 1 the model eval reposnce was in json , we did 10 trials , we slight chnages in eah trial and found out trial 1 was the best but now it has lost , do to diff exp , so we have to also tracks of these things too

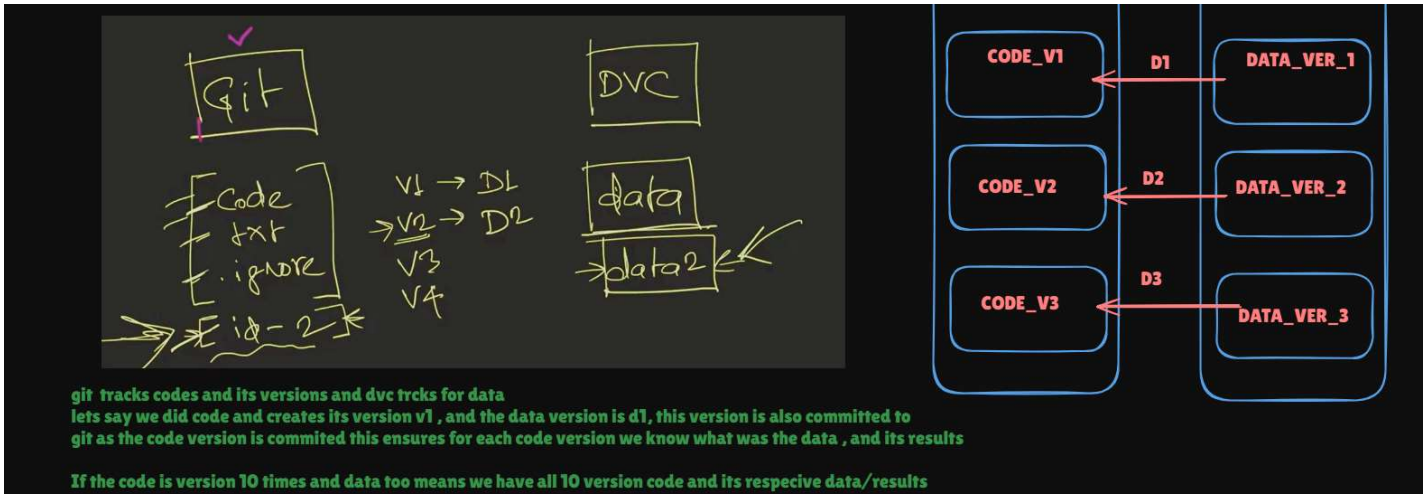
HOW?

EXAMPLE



HOW GIT AND DVC WORKS





✅ WHAT is DVC?

DVC = **Data Version Control**

It is a tool used in Machine Learning projects to:

- Track datasets
- Track ML models
- Track intermediate artifacts
- Track pipeline steps
- Reproduce experiments
- Share large files easily

Simple line:

> **DVC = Git for data, models, and ML pipelines.**

✅ HOW does DVC work? (Simple Explanation)

DVC works in **three simple ways**:

---

1 DVC tracks your large files without storing them in Git

Example:

`data.csv` is 5GB → Git cannot store it.

DVC does this:

- Stores metadata in Git (`data.csv.dvc`)
- Stores the real file in **remote storage** (Google Drive, AWS S3, local storage, etc.)

So Git remains fast.

---

2 DVC builds a pipeline where each stage has inputs and outputs (artifacts)

Example:

Auto

```
raw_data → clean_data → features → model → evaluation
```

Each output is an **artifact**.

DVC creates a file called **dvc.yaml** to track:

- Which code step produces which artifact
- Which input file was used
- Which output file was generated

This means:
If you change the data or code → only the affected steps rerun.

---

3 DVC ensures full reproducibility

With one command:

Auto

```
dvc repro
```

DVC can:

- Rebuild the entire pipeline
- Using the exact same data
- With exact versions
- And reproduce the same model

No confusion. No guessing.

⭐ SIMPLE SUMMARY

Here is DVC in two points:

✓ WHAT is DVC?

A tool to version **data**, **models**, and **ML pipeline artifacts** just like Git versions code.

✓ HOW does it work?

- Tracks large files with metadata
- Stores real data in remote storage
- Builds pipelines with clear inputs/outputs
- Reproduces everything with one commands

All **code versions can point to the same data version** if the dataset didn't change.

Here's the breakdown:

---

1 Data Version (immutable)

- Example: `data_v1`
- Stored once in remote storage
- Never overwritten

2 Code Versions (Git commits)

- Example: `code_v1` , `code_v2` , … `code_v10`
- Each represents a change in code, parameters, or pipeline logic

3 Artifacts

- Each run produces a **model or output artifact**
- Linked to **both the code version and the data version used**

Visual Example:

Auto

```
data_v1
    ├─ code_v1 → model_v1
    ├─ code_v2 → model_v2
    ├─ code_v3 → model_v3
```

- All **10 code versions** can point to the **same data_v1**
- Only **model artifacts** differ per code version

✅ Key point

DVC is smart: it **doesn't duplicate the data**.
It tracks **which code + data produced which artifact**, so you can reproduce any experiment.